Arquitectura de PromptGuard

Resumen simple para una primera lectura del código

PromptGuard es una librería que funciona como un **guardia de seguridad entre un modelo de inteligencia artificial y las herramientas de una aplicación**.

Antes de que una acción propuesta por el modelo se ejecute, PromptGuard revisa si está permitida.

```
Modelo de IA
    ↓
Propone usar una herramienta
    ↓
PromptGuard revisa la llamada
    ↓
Permite, bloquea, pide confirmación o deriva a una persona
    ↓
La herramienta se ejecuta —o no—
```

Las dos partes principales

```
src/main/java/ar/edu/utn/frsf/promptguard/
├── core/
└── spring/
```

1. Paquete `core`: el cerebro de seguridad

El paquete `core` contiene la lógica principal de PromptGuard. Esta parte no depende directamente de Spring.

Su trabajo es responder: *"¿Esta herramienta puede ejecutarse con estos argumentos y este contexto?"*

El archivo más importante

El componente central es `EvaluadorAcciones.java`. Su método principal es:

```
decidir(LlamadaHerramienta llamada,
ContextoSolicitud contexto)
```

Recibe la herramienta que el modelo quiere usar, sus argumentos, el contexto de la conversación y la política de seguridad. Devuelve una decisión:

- `PERMITIR`
- `BLOQUEAR`
- `CONFIRMAR`
- `DERIVAR`

¿Qué revisa `EvaluadorAcciones`?

C0: ¿La herramienta está registrada?

Si la herramienta no está en la política, se bloquea. Esto evita que el modelo pueda inventar o usar herramientas no autorizadas.

C1: ¿El asistente tiene permiso?

Una herramienta puede estar permitida para el asistente, prohibida o reservada para una persona.

```
reembolsar → REQUIERE_HUMANO
enviarEmail → PROHIBIDA
consultarSaldo → ASISTENTE
```

C2: ¿El cliente pidió esa acción?

Comprueba si el usuario realmente pidió algo relacionado con la herramienta. Si el usuario nunca pidió esa acción, PromptGuard puede bloquearla o pedir confirmación.

C3: ¿Los argumentos tienen un origen válido?

Comprueba de dónde salió cada argumento. Un valor puede venir de lo que escribió el usuario, una herramienta confiable o una lista blanca. Un dato que apareció solamente en un documento recuperado por

RAG no se considera suficiente para ejecutar una acción sensible.

C4: ¿El contexto es confiable?

Si la herramienta es sensible y la información proviene de documentos no confiables, PromptGuard puede pedir confirmación humana.

```
Documento externo + herramienta sensible → CONFIRMAR
```

2. Paquete `spring`: la conexión con la aplicación

El paquete `spring` conecta el núcleo de seguridad con Spring Boot y Spring AI. Esta parte integra PromptGuard automáticamente en una aplicación existente.

Autoconfiguración

`PromptGuardAutoConfiguration.java` registra automáticamente los componentes necesarios. La aplicación solo necesita agregar la dependencia y configurar la política.

Descubrimiento de herramientas

`PromptGuardChatClientCustomizer.java` busca las herramientas disponibles en la aplicación. Puede encontrar métodos anotados con `@Tool`, objetos `ToolCallback` y proveedores de herramientas.

Protección de herramientas

`ProtectedToolCallback.java` envuelve cada herramienta para que su ejecución pase primero por PromptGuard.

```
Llamada a la herramienta
    ↓
ProtectedToolCallback
    ↓
PromptGuardRuntime
    ↓
EvaluadorAcciones
    ↓
Decisión
```

```
    ↓
Ejecutar o bloquear
```

El runtime

`PromptGuardRuntime.java` coordina todo el proceso:

1. Recibe la llamada del modelo.
2. Lee el nombre de la herramienta y sus argumentos.
3. Obtiene el contexto de la conversación.
4. Llama a `EvaluadorAcciones`.
5. Registra la decisión.
6. Ejecuta o rechaza la herramienta.

En otras palabras, `EvaluadorAcciones` decide y `PromptGuardRuntime` ejecuta esa decisión.

El contexto de la solicitud

`ContextoSolicitud.java` guarda la información necesaria para evaluar una llamada:

- Los mensajes escritos por el usuario.
- Los documentos recuperados por el sistema RAG.
- La procedencia de esos documentos.
- Los resultados de herramientas confiables.

Configuración

La configuración se lee desde `application.yaml` o `application.properties`, agrupada bajo `promptguard`.

```
promptguard:
  modo: enforce
  herramientas:
    consultarSaldo:
      autoridad: asistente
    enviarEmail:
      autoridad: prohibida
    reembolsar:
      autoridad: requiere_humano
```

Si una herramienta no aparece en la configuración, se bloquea automáticamente.

Confirmaciones humanas

Cuando una acción necesita confirmación, se guarda en `AccionesPendientes.java`. La acción recibe un identificador, conserva sus argumentos originales, expira después de un tiempo y solo puede ejecutarse una vez.

Auditoría

PromptGuard registra qué herramienta se intentó usar, qué decisión se tomó, qué regla la bloqueó, si finalmente se ejecutó y cuánto tardó la evaluación. Los datos sensibles se ocultan antes de registrarse.

Resumen de responsabilidades

Componente	Responsabilidad
`EvaluadorAcciones`	Decide si una llamada está permitida.
`Politica`	Define las reglas de cada herramienta.
`ContextoSolicitud`	Guarda los datos de la conversación y del RAG.
`PromptGuardRuntime`	Coordina la evaluación y la ejecución.
`ProtectedToolCallback`	Intercepta las llamadas a herramientas.
`PromptGuardAdvisor`	Agrega información del RAG al contexto.
`PromptGuardAmbitoAdvisor`	Abre el contexto de la solicitud.
`AccionesPendientes`	Guarda acciones que esperan confirmación.
`RegistroAuditoria`	Registra las decisiones.

`PromptGuardProperties`	Representa la configuración YAML.

Cómo seguir el código

1. `EvaluadorAcciones.java`: entender las reglas C0 a C4.
2. `Decision.java`: ver los resultados posibles.
3. `ContextoSolicitud.java`: entender qué información usa el evaluador.
4. `PromptGuardRuntime.java`: ver cómo se conecta la decisión con la ejecución.
5. `ProtectedToolCallback.java`: ver cómo se interceptan las herramientas.
6. `PromptGuardAutoConfiguration.java`: entender cómo Spring registra todo automáticamente.

> **Idea principal:** el paquete `core` decide qué está permitido y el paquete `spring` conecta esa decisión con las herramientas reales de la aplicación.